



2026-1 텍스트마이닝

과제_3 Word2Vec_구현

세종대학교
데이터사이언스 학과
학번: 22011831
이름: 김형규



제출 관련 공지사항

- 제출 관련 공지사항 슬라이드는 제출 시 삭제
- 과제 제출일
 - 과제3: 5월 7일 (목) 오후 11시 59분
- 과제 제출물
 1. 코드 완성한 ipynb 파일
 - 파일명: “과제_3_Word2Vec_구현_학번000000_이름000.ipynb”
 - 파일명이 위 형식에 맞지 않을 경우 감점
 2. 현 PPT 양식을 활용하여 각 Step 별로 진행 상황을 기록한 ppt 파일을 PDF로 변환 후 제출
 - 파일명: “과제_3_Word2Vec_구현 _학번000000_이름000.pdf”
 - 파일명이 위 형식에 맞지 않을 경우 감점



Step 1

```
corpus = [  
    "he is a king",  
    "she is a queen",  
    "he is a man", "she is a woman",  
    "seoul is korea capital",  
    "tokyo is japan capital",  
    "paris is france capital"  
]  
  
words = " ".join(corpus).split()  
word_list = list(set(words))  
word_dict = {w: i for i, w in enumerate(word_list)}  
vocab_size = len(word_dict)
```

```
vocab_size: 15  
word_dict: {'japan': 0, 'korea': 1, 'he': 2, 'tokyo': 3,  
            'france': 4, 'queen': 5, 'seoul': 6, 'is': 7, 'paris': 8,  
            'she': 9, 'man': 10, 'king': 11, 'a': 12, 'capital': 13,  
            'woman': 14}
```

코드 작성 이유

1. Word2Vec 학습은 단어를 정수 인덱스로 매핑한 사전이 필수이므로, 코퍼스 토큰화하고 set()으로 중복을 제거해 word_dict / number_dict를 만들었다.
2. 이후 단계의 원-핫 벡터 차원과 출력층 크기가 모두 vocab_size에 의존하므로, 사전 구축이 가장 먼저 수행되어야 한다.

분석 결과

1. 총 어휘 수는 15개(vocab_size=15)로 매우 작은 toy 코퍼스이다. 작은 어휘는 학습 시간을 짧게 해주지만, 의미 관계가 형성되려면 동일 패턴이 여러 번 등장해야 한다.
2. 코퍼스 내에 'king/queen', 'man/woman', 'seoul/tokyo/paris' 같은 의미적 짝이 의도적으로 배치되어 있어, Skip-gram이 잘 학습되면 시각화에서 군집이 형성될 것이라 기대된다.
3. set() 자료형은 순서가 보장되지 않아 word_dict 인덱스가 실행마다 달라질 수 있는데, 이는 학습 결과 시각화 좌표에는 영향이 없고 단순히 인덱스 매핑만 바뀐다.



Step 2

```
window_size = 1
skip_grams = []
for sentence in corpus:
    tokens = sentence.split()
    for i in range(len(tokens)):
        target = word_dict[tokens[i]]
        for j in range(max(0, i - window_size),
                        min(len(tokens), i + window_size + 1)):
            if i != j:
                context = word_dict[tokens[j]]
                skip_grams.append([target, context])
```

```
len(skip_grams): 42
```

```
skip_grams[0] (Target, Context):
```

```
[[2, 7], [7, 2], [7, 12], [12, 7], [12, 11]]
```

코드 작성 이유

1. Skip-gram은 중심 단어로 주변 단어를 예측하므로 (target, context) 쌍 자체가 학습 데이터가 된다. window_size 내의 인덱스를 순회해 자기 자신을 제외한 모든 주변 단어를 짝지어 주었다.
2. tokens[j]를 word_dict로 매핑해 인덱스 형태로 저장한 이유는 CrossEntropyLoss가 정답 라벨로 정수 인덱스를 요구하기 때문이다.

분석 결과

1. window_size=1이라는 좁은 윈도우에서도 7개 문장만으로 42개의 학습 쌍이 생성됐다. 같은 문장 안에서 (target, context)와 (context, target)이 모두 들어가 양방향 학습이 자연스럽게 이뤄진다.
2. 예: 'he is a king' → (he,is)(is,he)(is,a)(a,is)(a,king)(king,a). 'is'와 'a'가 거의 모든 문장에 등장해 학습 쌍에서 가장 자주 나타나며, 이는 일반적 의미를 갖는 단어가 어휘 공간 중심부에 위치하는 결과로 이어진다.
3. 윈도우가 작으면 직접 인접 단어만 학습하므로 의미 군집은 명확해지지만, 멀리 떨어진 단어 간 관계(예: 같은 문장의 첫 단어와 마지막 단어)는 직접적으로 학습되지 않는다.



Step 3

```
class Word2Vec(nn.Module):
    def __init__(self, vocab_size, embedding_size):
        super().__init__()
        self.W = nn.Linear(vocab_size, embedding_size, bias=False)
        self.WT = nn.Linear(embedding_size, vocab_size, bias=False)
    def forward(self, X):
        hidden_layer = self.W(X)
        output_layer = self.WT(hidden_layer)
        return output_layer

embedding_size = 2
model = Word2Vec(vocab_size, embedding_size)
```

코드 작성 이유

1. Skip-gram은 본질적으로 $V \rightarrow D \rightarrow V$ 의 두 단 선형 변환이므로, bias 없는 두 개의 `nn.Linear`만으로 구현 가능하다. W 는 입력층-은닉층(임베딩) 행렬, WT 는 은닉층-출력층 행렬이다.
2. 임베딩 시각화를 2차원 평면에서 직접 확인하려고 `embedding_size=2`로 설정했다. 실 사용에서는 보통 100~300차원을 쓴다.

분석 결과

1. 은닉층에 비선형 활성화가 없는 점이 핵심이다. 이는 `Word2Vec`이 행렬 분해(matrix factorization) 관점으로 해석될 수 있는 이유와 직결된다.
2. 최종 임베딩으로 사용하는 것은 W 의 가중치 행렬이며, WT 는 학습 보조 용이다. 모델 파라미터 수는 $V \cdot D + D \cdot V = 15 \cdot 2 + 2 \cdot 15 = 60$ 개로 매우 작다.
3. `bias=False`로 두는 이유는 원-핫 입력 + 단순 lookup 구조에서 `bias`가 단어별 임베딩 의미를 흐리기 때문이다.



Step 4

```
critterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1)

def get_one_hot(word_idx, vocab_size):
    x = torch.zeros(vocab_size)
    x[word_idx] = 1.0
    return x
```

One-hot vector for index 0:

```
tensor([1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

코드 작성 이유

1. 출력은 어휘 전체에 대한 점수이므로 다중 클래스 분류로 보고 CrossEntropyLoss를 선택했다. 이 손실은 내부에 softmax가 포함되어 별도 softmax 호출이 필요 없다.
2. Adam 옵티마이저는 작은 코퍼스에서도 학습률을 자동 조절해 빠르게 수렴하므로 채택했다.
3. get_one_hot()으로 단어 인덱스를 V차원 원-핫 벡터로 변환해야 nn.Linear의 입력 형태와 맞는다.

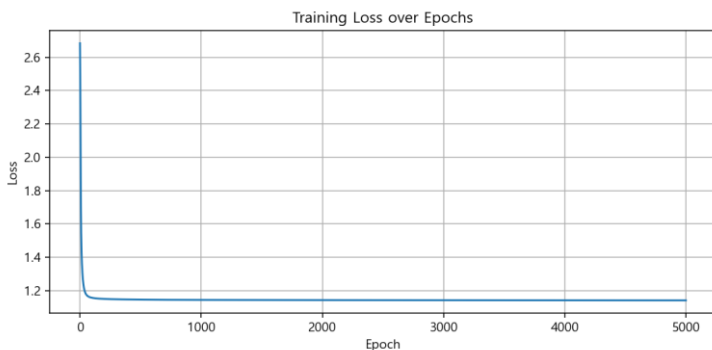
분석 결과

1. 원-핫 벡터를 W에 통과시키는 연산은 사실상 W의 해당 단어 행을 그대로 가져오는 'lookup'과 동일하다. 행렬곱처럼 보이지만 실제로는 임베딩 테이블 조회이다.
2. 실무에서는 이 비효율을 피하려고 nn.Embedding을 쓰지만, 본 과제는 원-핫 → Linear의 흐름을 명시적으로 보여주기 위한 교육적 구현이다.
3. 출력에서 인덱스 0 위치만 1.0인 길이 15짜리 벡터가 잘 만들어졌음을 확인했다.



Step 5

```
epochs = 5000
losses = []
for epoch in range(epochs):
    loss_val = 0
    for target, context in skip_grams:
        x = get_one_hot(target, vocab_size)
        y_true = torch.tensor([context], dtype=torch.long)
        optimizer.zero_grad()
        y_pred = model(x)
        loss = criterion(y_pred.unsqueeze(1), y_true)
        loss.backward()
        optimizer.step()
        loss_val += loss.item()
    losses.append(loss_val / len(skip_grams))
```



코드 작성 이유

1. 각 (target, context) 쌍마다 forward → loss → backward → step 흐름을 반복해 W, WT 가중치를 업데이트한다. zero_grad()로 이전 gradient를 초기화하지 않으면 누적되어 학습이 망가지므로 매 스텝에서 호출했다.
2. epoch마다 평균 loss를 기록해 학습 진행 상황을 확인하고 시각화하기 쉽도록 했다.

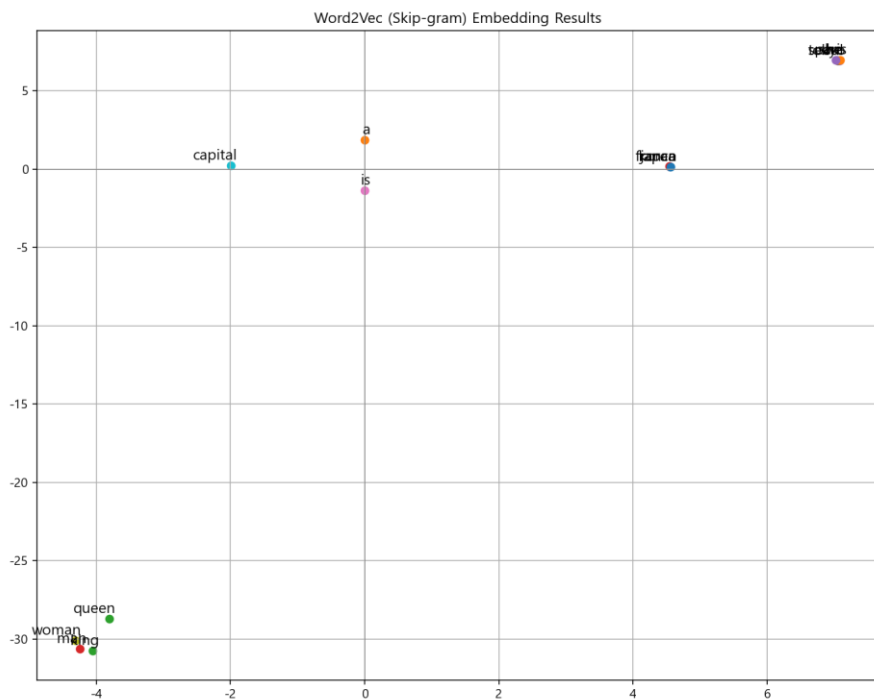
분석 결과

1. 초기 loss는 약 2.71에서 시작해 1000 epoch 만에 1.14 부근으로 빠르게 떨어진 뒤 거의 평탄해진다. 모델이 toy 코퍼스의 가능한 (target, context) 분포를 거의 다 학습했음을 의미한다.
2. loss가 0이 아니라 1.14 근처에서 멈추는 이유는, 같은 target이 다수의 다른 context를 가지므로(예: 'is' → he/she/a/korea/japan...) 한 번의 예측으로 모든 정답을 동시에 맞출 수 없기 때문이다. 이는 본질적인 cross-entropy 하한이다.
3. saddle 없이 매끄럽게 수렴하는 모습은 Adam의 적응형 학습률 덕분이며, SGD를 사용하면 더 흔들리고 수렴 속도도 느려질 가능성이 크다.



Step 6

```
W_embed = model.W.weight.data.numpy().T
plt.figure(figsize=(10, 8))
for i, word in enumerate(word_list):
    x, y = W_embed[i][0], W_embed[i][1]
    plt.scatter(x, y)
    plt.annotate(word, (x, y), xytext=(5, 2),
                  textcoords='offset points', fontsize=12)
plt.title("Word2Vec (Skip-gram) Embedding Results")
plt.show()
```



코드 작성 이유

1. 학습된 W의 행 벡터가 곧 단어의 임베딩이므로 model.W.weight.data를 numpy 배열로 변환했다. .T를 취해 (V, D) 형태로 만든 뒤 단어별 좌표로 사용했다.
2. embedding_size=2이므로 별도 차원 축소(PCA, t-SNE) 없이 그대로 산점도로 시각화 가능하다.

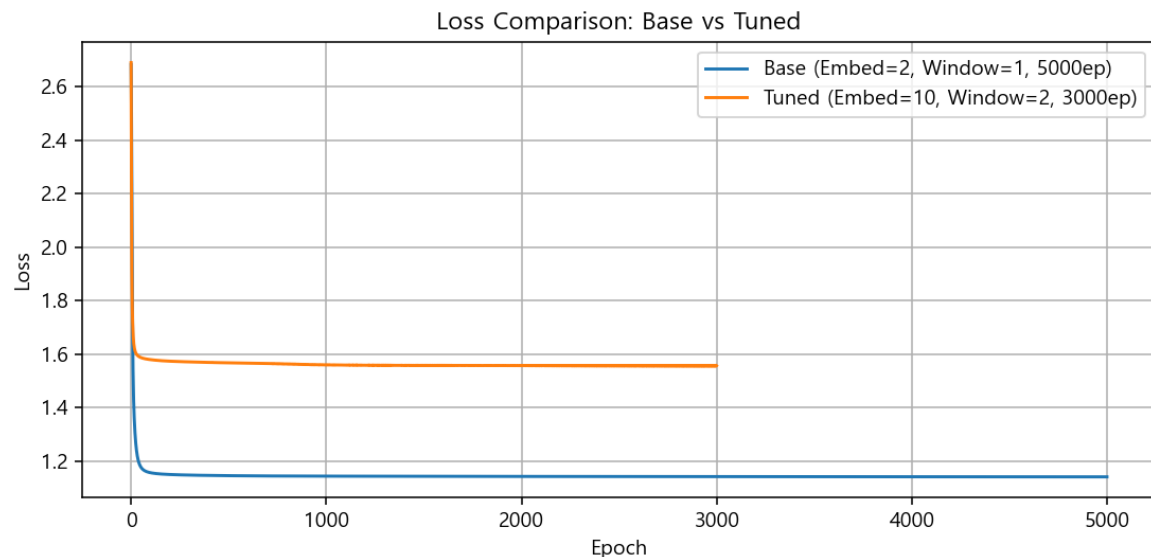
분석 결과

1. 결과 그래프에서 의미적으로 비슷한 단어가 같은 영역에 군집을 이루었다. 좌측 하단에 king/queen/man/woman 같은 사람·성별 관련 단어가 모였고, 우측 상단에 도시명(seoul/tokyo/paris)과 대명사(he/she)가, 우측 중앙에 국가명(korea/japan/france)이 모여 있다.
2. 조사·관사 역할의 'is', 'a'는 원점 근처에 자리잡고 있는데, 거의 모든 문장에 등장해 특정 카테고리에 치우치지 않은 결과로 해석된다.
3. 이러한 군집화는 Skip-gram이 분포 가설(distributional hypothesis: 비슷한 문맥에서 등장하는 단어는 비슷한 의미를 가진다)을 잘 반영했다는 직접적 증거이다.



Step 7

```
tuning_embedding_size = 10
tuning_window_size = 2
tuning_epochs = 3000
# (2) (2) (window_size 2)
tuning_skip_grams = []
# (2) skip_grams (2) (2) (2) (2) (2) (2) (2) (2) (2) (2)
tuning_model = Word2Vec(vocab_size, tuning_embedding_size)
tuning_optimizer = optim.Adam(tuning_model.parameters(), 0.1)
# (2) (2) (2) Loss (2) (2) (2)
```



코드 작성 이유

1. embedding_size를 2→10으로 늘려 표현력을 확장하고, window_size를 1→2로 늘려 더 넓은 문맥을 학습 신호로 사용했다. 두 변화가 학습 양상에 미치는 영향을 함께 관찰하려는 의도이다.
2. epoch는 5000→3000으로 줄여 학습 비용 변화도 같이 점검했다.

분석 결과

1. window_size=2로 확장하니 학습 쌍이 42개 → 70개로 약 +67% 늘었다. 한 target 단어가 예측해야 하는 context의 종류도 많아져 per-pair cross-entropy loss는 base(1.14)보다 오히려 높은 1.55 부근에서 수렴했다.
2. loss 값이 더 크다고 해서 모델 품질이 나쁜 것은 아니다. 윈도우가 커질수록 같은 target이 더 다양한 context를 예측해야 하므로 이론적 cross-entropy 하한 자체가 올라간다. 즉 loss 절댓값이 아니라 학습 곡선의 안정성과 임베딩 품질로 평가해야 한다.
3. embedding_size 2→10 증가로 의미 표현 공간이 더 풍부해져 단어 간 미세한 차이까지 표현 가능해진다는 trade-off가 있다. 다만 시각화는 직접 불가능해지므로 PCA 등 차원 축소가 필요하다.
4. 실 응용에서는 시각화보다 임베딩 품질이 중요하므로 embedding_size 100~300이 일반적이며, window_size도 코퍼스 특성과 도메인에 따라 조절한다.